

An Analysis of Streaming Deep Reinforcement Learning for Adaptive Continual Learning in Robotics

Teeratham Vitchutripop¹, Alyssa Quarles¹, Wenhe Zhang¹, Richard Xue¹, Daniel Rakita¹

Abstract—Over the course of a lifetime, robots may encounter novel scenarios unaccounted for in its original training that result in performance degradation. One common approach to mitigating this issue is to further grow the offline training dataset in hopes of producing a policy robust to these changes. In contrast, biological learning occurs moment-to-moment via a *stream of experience*, unlike the predominantly batch-based and offline nature of deep learning. Although recent works show the feasibility of *stream-based deep reinforcement learning*, where updates use only the latest experience, none have shown it to be a viable continual learning framework for adapting robotic policies to unseen changes. In this paper, we present the first analysis of streaming deep reinforcement learning for adaptive continual learning in robotics. In particular, we show that, following an initial pretraining phase, streaming deep RL can enable a robot to successfully adapt to unforeseen changes to itself, its environment, or goals. Our experiments within quadruped locomotion demonstrate that a deep neural network robotic policy with certain optimizers and plasticity loss mitigation techniques can successfully leverage domain task knowledge from its pretraining to quickly adapt online to diverse changes via stream learning, outperforming batch-based on-policy methods and improving task success rates by up to 90% over the pretrained policy. We conclude with a discussion on the limitations of our work and its implications for the future of continual robot learning.

I. INTRODUCTION

The capacity for robots to continuously learn from their own stream of experience is attractive both for its parallels to biological learning and the fast adaptation it could enable. Even as computational resources expand, the “world” in a learning problem remains orders of magnitude larger and more complex than the agent, making it impractical to assume pretrained models can anticipate every scenario [9]. This observation motivates an alternative paradigm: designing agents that may be smaller and more resource constrained, but can continually learn online via a *stream of experience* (i.e., stream learning) to meet the decision-making problems of the present. The strategy mirrors how humans and animals, while limited by the size and energy of a biological brain, still possess a remarkable capacity to learn moment-to-moment through experience [7].

Despite its promise, stream learning contrasts with the dominant deep learning literature in robotics. State-of-the-art approaches, e.g., vision-language action models [8], large behavior models [18], and other imitation learning approaches, generally assume batch-based offline learning focused on scaling model size and datasets. Likewise, deep reinforcement learning (RL) has moved away from classical

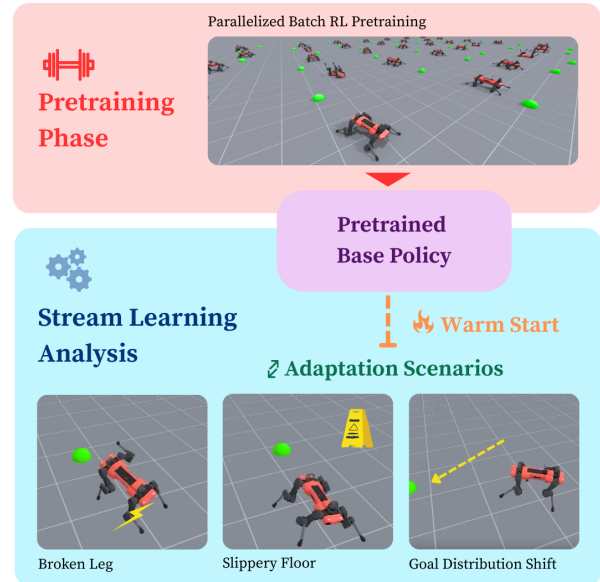


Fig. 1: An overview of our analysis on streaming deep reinforcement learning for continual adaptation in robotic systems. We examine if pretrained RL policies are able to adapt online in a stream-fashion to a set of scenarios that introduce a sudden change to the robot, its environment, or goals.

stream-based RL [20], favoring batch updates and replay buffers to improve stability and sample efficiency [1].

Recently, however, stream learning has begun to re-emerge within deep RL [6, 22]. This progress has been primarily spurred by advances in optimizers and by scaling and normalization techniques inspired by neural network plasticity research, which examines how networks maintain performance while learning from new data over time [13]. Despite this exciting progress, no prior work has established whether stream learning using these advancements can serve as a viable continual learning framework for adapting robotic policies to unseen changes.

In this paper, we present the first analysis of streaming deep reinforcement learning for adaptive continual learning in robotics (Fig. 1). Our analysis particularly focuses on adapting pretrained policies, leveraging the advancements made in parallelized batch RL to establish a base policy. We begin by studying how streaming deep RL can enable successful adaptation to unforeseen changes to the robot, its environment, or goals within a quadruped locomotion domain. More specifically, we examine the role that optimizers and plasticity loss mitigation techniques play in enabling a robot to recover from abrupt changes and maintain robust performance over time.

Our results suggest that pretrained deep RL policies can maintain performance after switching to stream learning and

¹Department of Computer Science, Yale University, New Haven, CT 06520, USA

successfully adapt to novel scenarios. We observe that deep neural networks remain sufficiently malleable and stable during online stream learning, allowing a robot to autonomously self-improve from its experiences by up to 90% in success rate over the base policy on certain adaptation scenarios. We identify that optimizer choice contributes significantly to stability and plasticity loss mitigation techniques like layer normalization [3] and continual backpropagation [4] can be helpful in enhancing adaptation performance and recovery speed. We conclude with a discussion on the limitations of our work and potential opportunities for future research stemming from our analysis.

We release our code as open source to enable the robotics community to evaluate, reproduce, and build upon our work.¹

II. RELATED WORKS

A. Batch and Stream Reinforcement Learning

Foundational RL algorithms such as TD, SARSA, and Actor-Critic [20] were designed for stream learning, updating policies incrementally from each experience without storage. Their scope, however, was largely limited to tabular settings. As neural networks gained wider adoption in RL for function approximation [15], batch-based practices from deep learning took hold to reduce instability and improve sample efficiency: replay buffers [1], offline RL [12], and GPU-enabled parallel simulation [21] all shifted the field away from streaming. More recent works show that normalization and scaling techniques, such as observation and layer normalization, can enable effective streaming deep RL [6, 22]. Vasan et al. [22] introduced a normalization-based policy gradient method with RG estimation, while Elsayed et al. [6] identified large step sizes as a key instability source and proposed bounded optimizers.

While Vasan et al. [22] successfully applied streaming deep RL to train robotic systems from scratch (e.g., manipulator reaching and Roomba navigation), tasks like quadruped locomotion are more challenging and inefficient to learn from scratch via a stream-setting. Our work leverages the advancements made in parallelized simulation and batch RL to bootstrap the policy via pretraining. Moreover, no prior work has explored streaming deep RL’s potential for adaptation and continual learning in robotics.

B. Plasticity in Deep Learning

Plasticity refers to a neural network’s ability to continue adapting as it encounters new data [13]. For continual and streaming deep RL, plasticity is crucial: policies must remain flexible under distribution shifts and ongoing updates. However, prior work shows that standard deep neural networks often lose this capacity over time, a phenomenon known as *plasticity loss* [4, 17, 16].

Several strategies have been proposed to counter plasticity loss, including reinitializing underused units [4], injecting new parameters [17], adjusting optimizers [16], and applying normalization layers [13]. These techniques highlight that

preserving plasticity is as much about stabilizing optimization as it is about maintaining network expressivity. Our work revisits this issue in the context of streaming deep RL, analyzing whether current methods allow pretrained policies to adapt reliably to new robotic scenarios, rather than just a single fixed task.

C. Continual Learning and Adaptation in Robotics

Continual learning is a machine learning paradigm where models adapt to shifts in data distributions and objectives over time [11]. In robotics, while some approaches enable adaptation of pretrained behavior cloning policies via residual RL [2] or dynamic rank adjustment [19], several proposed methods target RL policies directly. Lyu et al. [14] adapt pretrained RL policies with torque compensation to handle dynamic disturbances. Dong et al. [5] propose batch online RL, alternating between offline pretraining, real-world data collection, and retraining. In contrast, our work investigates whether streaming deep RL itself can serve as a foundation for continual adaptation in robotics. Although continual learning is also concerned with catastrophic forgetting (backward transfer), we limit the scope of our analysis to forward transfer, using prior knowledge for adaptation.

III. BACKGROUND

In this section, we provide background for concepts that will be relevant for our analysis in §IV.

A. Markov Decision Processes

We formulate the problem of solving a robotic task as an episodic Markov Decision Process (MDP), given by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the continuous state space, $\mathcal{A}$ is the continuous action space, $\mathcal{P}$ is the transition probabilities, $\mathcal{R}$ is the normalized dense reward function, and $\gamma \in [0, 1]$ is the discount factor. At each timestep t , the agent interacts with the environment by sampling an action A_t from a parameterized policy π_θ based on the current state S_t , $A_t \sim \pi_\theta(\cdot | S_t)$. The objective of the agent is to find a weight parameterization θ for π_θ that maximizes the expected return, defined as $J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t R(S_t, A_t) \right]$, where T is the episode horizon.

Through this work, we are specifically assessing a robot’s ability to adapt to scenarios where the transition dynamics $\mathcal{P}$ and reward function $\mathcal{R}$ of the underlying MDP abruptly change. Our setting is defined by a pair of MDPs in sequence $[M_0, M_1] = [(\mathcal{S}, \mathcal{A}, \mathcal{P}_0, \mathcal{R}_0, \gamma), (\mathcal{S}, \mathcal{A}, \mathcal{P}_1, \mathcal{R}_1, \gamma)]$, where M_0 is the same conditions as the pretraining phase and M_1 is an adaptation scenario.

B. Streaming Reinforcement Learning

Our analysis is focused on a subset of online RL approaches known as streaming deep RL. Unlike batch-based RL, at each timestep t , streaming deep RL algorithms, like Stream-AC [6], update the parameters of π_θ and V_ϕ by computing the TD error δ based on only the most recent transition. Theoretically, this setting should enable the agent to adapt faster to changes, as the policy is continuously

¹Link to GitHub repository.

updated at each step with new experience. However, stream learning is prone to training instabilities, as large parameter updates to θ or ϕ from a noisy TD error can destabilize the agent's policy. At the same time, overly conservative updates will cause the agent to fail to learn the task. These challenges highlight the need for special attention to controlling the size of updates and maintaining the networks' plasticity.

C. Optimizers for Stream Learning

In our analysis, we assess different optimization schemes for stream learning. Due to its popularity and ubiquity in deep learning, we use the Adam [10] optimizer as a baseline. Additionally, we evaluate two optimizers designed for streaming RL that de-emphasize large updates by bounding the effective step size: Overshooting-bounded Gradient Descent (ObGD), and Adaptive Overshooting-bounded Gradient Descent (AdaptiveObGD) [6].

ObGD and AdaptiveObGD use eligibility traces, i.e., short-term memory vectors of decayed parameter sensitivities, to assist with multi-step credit assignment. For ObGD, given $\bar{\delta} = \max(|\delta|, 1)$, eligibility trace z_t , step size α , and scaling factor κ , the step size is bounded and updated via the following rule: $\alpha \leftarrow \min(\frac{1}{\kappa \bar{\delta} \|z_t\|_1}, \alpha)$. The parameters θ and ϕ are then updated by $w \leftarrow w + \alpha \delta z_w$, where w is the weights. AdaptiveObGD slightly modifies the ObGD optimizer to be more like Adam by incorporating a second moment estimate.

D. Methods for Combating Plasticity Loss

Our analysis also assesses the efficacy of plasticity loss mitigation techniques for stream learning. In particular, we evaluate two approaches: layer normalization (LN) [3] and continual backpropagation (CBP) [4].

Layer normalization (LN) normalizes values across the feature dimension of the intermediate representations for a single sample during a forward pass within a neural network.

Continual backpropagation (CBP) reactivates the least used units in a neural network determined via a contribution utility metric. A unit's contribution utility $\mathbf{u}_l[i]$ is the sum of the utility of its outgoing connections and is maintained as a running average with a decay rate, η . To accommodate for our Tanh activation function, we replace the activation magnitude scaling used in the original algorithm, $|\mathbf{h}_{l,i,t}|$, with the gradient capacity, $(1 - \mathbf{h}_{l,i,t}^2)$, resulting in the following contribution utility:

$\mathbf{u}_l[i] = \eta \times \mathbf{u}_l[i] + (1 - \eta) \times (1 - \mathbf{h}_{l,i,t}^2) \times \sum_{k=1}^{n_{l+1}} |\mathbf{w}_{l,i,k,t}|$, where $\sum_{k=1}^{n_{l+1}} |\mathbf{w}_{l,i,k,t}|$ is the sum of weights extending from this particular unit to the n_{l+1} units in layer $l + 1$.

IV. ANALYSIS

Our analysis is a series of experiments conducted within quadruped locomotion that 1) demonstrate the potential capabilities of streaming deep RL for enabling continual robotic adaptation and 2) analyze how the choice of certain optimizers or plasticity loss mitigation techniques can affect adaptation performance.

A. Experimental Setup

Our experiments model the non-stationary MDP $[M_0, M_1]$, where M_0 is the 500,000 step warm-start period on the base task and M_1 is the adaptation scenario lasting 1.5 million steps. During M_1 , we obtain the max and mean success rates from 50 episode evaluations that occur every 10,000 steps during training. Furthermore, to evaluate the stability of a policy during M_1 , we use the mean absolute successive difference (MASD) metric formalized as $\frac{1}{N-1} \sum |x_n - x_{n-1}|$, where N is the total number of sampled evaluations and x_n is the success rate at the n th evaluation. Results can be found in Table I, Table II, and Fig. 2.

Training runs were conducted with 5 random seeds. All comparative methods, except AdaptiveObGD + LN, use the same set of pretrained weights from a successful parallelized batch PPO training run on the base task. AdaptiveObGD + LN uses a set of weights that incorporates layer normalization in the pretraining. A 4-layer multi-layer perceptron with size 256 hidden layers and a Tanh activation function were used for the actor-critic networks in all experiments.

B. Task Environment and Adaptation Scenarios

Our analysis is performed within the AnymalC-Reach-v1 task environment in ManiSkill3 [21], which requires an Anymal C quadrupedal robot to walk to a goal position in the environment. The goal is randomized within a certain distribution in front of the robot. State space is used for observations and actions are represented by the change in joint positions. We use the provided normalized dense reward function, which includes a basic distance-to-goal reward and penalties to shape the robot's walking behavior.

We developed 3 adaptation scenarios that test the robot's ability to adapt to changes to its embodiment, environment, and goals. **(1) Broken Leg:** We zero-mask 3 values of the action space to simulate the back left leg of the robot being stuck. **(2) Slippery Floor:** We change the static and dynamic friction coefficients of the ground in the environment from 0.3 to -1.8. **(3) Goal Shift:** We offset the distribution of goal positions from the original centered at (0,0), to (0,3.0).

C. Evaluated Methods

We analyze the impact of different optimizers and plasticity loss mitigation mechanisms with a total of 5 unique variants of Stream-AC. To compare the effects of using different optimizers, 3 variants use Adam, ObGD, and AdaptiveObGD. The Adam variant has a TD(0) Advantage Actor-Critic update, while ObGD and AdaptiveObGD use a TD(λ) update. To compare plasticity loss mitigation mechanisms, we have 2 additional variants with the AdaptiveObGD optimizer: one that includes continual backpropagation (CBP) and another that performs layer normalization (LN). Note that CBP is only incorporated during the stream learning phase. Off-the-shelf default hyperparameters were used for ObGD and AdaptiveObGD variants, and thorough hyperparameter tuning was performed for the Adam variant to prevent performance collapse during the warm-start.

Table 1. Average Success Rates $\pm$ STD Across 5 Seeds on Adaptation Scenarios.

Method	Broken Leg		Goal Shift		Slippery Floor	
	Max	Mean	Max	Mean	Max	Mean
Adam	0.188 ± 0.027	0.080 ± 0.017	0.108 ± 0.018	0.026 ± 0.002	0.032 ± 0.011	0.003 ± 0.002
ObGD	0.936 ± 0.030	0.644 ± 0.032	0.808 ± 0.069	0.306 ± 0.085	0.420 ± 0.191	0.113 ± 0.056
AdaptiveObGD	0.968 ± 0.011	0.747 ± 0.019	0.656 ± 0.092	0.181 ± 0.074	0.368 ± 0.044	0.097 ± 0.028
AdaptiveObGD + CBP	0.920 ± 0.025	0.697 ± 0.034	0.604 ± 0.059	0.246 ± 0.066	0.344 ± 0.173	0.147 ± 0.088
AdaptiveObGD + LN	0.864 ± 0.026	0.538 ± 0.011	0.848 ± 0.034	0.436 ± 0.046	0.676 ± 0.107	0.278 ± 0.026
Batch PPO (No Warm-Start)	0.492 ± 0.129	0.163 ± 0.044	0.088 ± 0.018	0.014 ± 0.005	0.040 ± 0.014	0.002 ± 0.001
Batch PPO	0.412 ± 0.154	0.111 ± 0.047	0.060 ± 0.020	0.008 ± 0.004	0.004 ± 0.009	0.000 ± 0.000
Pretrained Policy	0.060	-	0.000	-	0.000	-
Pretrained Policy (with LN)	0.000	-	0.000	-	0.000	-

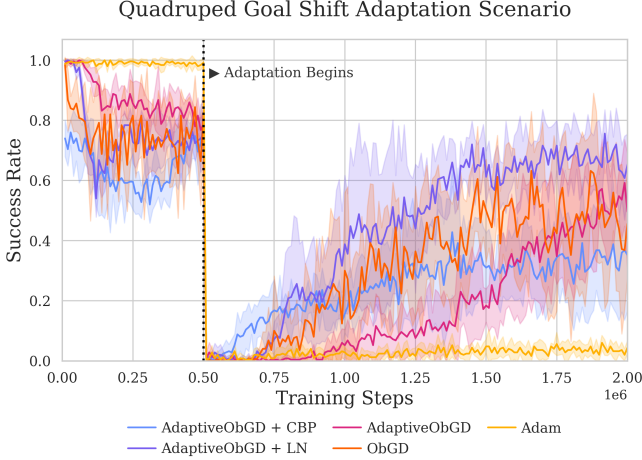


Fig. 2: The performance of our different Stream-AC variants on the quadruped goal shift adaptation is shown. AdaptiveObGD + LayerNorm outperforms other variants. ObGD is able to achieve a higher performance than AdaptiveObGD, but at the cost of stability.

D. Quantitative Results

Table 2. Average MASD $\pm$ STD (lower is more stable)

Method	Broken Leg	Goal Shift	Slippery Floor
ObGD	0.124 ± 0.01	0.099 ± 0.02	0.059 ± 0.02
AdaptiveObGD	0.067 ± 0.00	0.046 ± 0.01	0.032 ± 0.01

As shown in Table I, of the three optimizers tested, ObGD and AdaptiveObGD significantly outperformed Adam on all scenarios, featuring both a high max and mean success rate. This result aligns with the literature suggesting that Adam is not well-suited for stream learning [6], demonstrating the value of step size bounding mechanisms and eligibility traces. In Table II, we find that ObGD consistently has a higher MASD than AdaptiveObGD on average across all adaptation scenarios, indicating that ObGD is comparatively less stable than AdaptiveObGD. We can visually observe this instability in Fig. 2 as well. This result suggests that having an adaptive optimizer with a second moment estimate can improve performance stability, although this may come at the cost of volatile performance spikes that can lead to a higher max and mean success rate.

With the plasticity loss mitigation techniques added to AdaptiveObGD, we see that including layer normalization results in the best performance on the goal shift and slippery floor adaptations of all Stream-AC variants. This conforms with prior work suggesting that LN is beneficial for plasticity and training stability [13, 6].

Although there were minimal benefits to max success rate, AdaptiveObGD + CBP outperforms regular AdaptiveObGD in mean success rate on the goal shift and slippery floor scenarios. We interestingly observe in Fig. 2 that, despite having the most significant performance degradation during the warm-start, AdaptiveObGD + CBP is the first method to meaningfully recover after the goal shift, which may explain the higher mean success rate compared to regular AdaptiveObGD. This result highlights the potential of directly injecting diversity into neural networks, but also the long-standing challenge of the plasticity-stability tradeoff.

E. Comparison to Batch-Based Approaches

Surprisingly, our results shown in Table I show that the top four Stream-AC variants significantly outperform batch PPO on all adaptation scenarios in max and mean success rate. Because the literature suggests that Adam does not handle non-stationarity well [13], we compare our Stream-AC variants against a batch-based PPO that does not undergo the warm-start period. Although this improves batch PPO’s performance across all scenarios, thus corroborating Adam’s deficiencies in non-stationary settings, batch-based PPO still underperforms in success compared to our top performing Stream-AC variants on all tasks. These results underscore that streaming deep RL can be competitive against batch-based deep RL in adaptation performance.

V. DISCUSSION

In this paper, we presented the first analysis of streaming deep reinforcement learning for adaptive continual learning in robotics. Our analysis was centered on using streaming deep RL to adapt pretrained policies online to abrupt changes to the robot’s embodiment, environment, and goals. We show through our experiments that, with careful design considerations to optimizer choice and plasticity loss mitigation, streaming deep RL is effective in adapting pretrained policies to change, outperforming even common batch-based deep RL approaches like PPO.

Our work, however, is currently limited to simulation and abrupt single-stage adaptations. Future work can explore the potential of adaptive streaming deep RL for real-world on-device learning over extended deployment and with more complex, gradual multi-stage adaptation scenarios. We consider this work the beginning of a broader effort to develop more stable and performant streaming deep RL methods that enable robots to learn continuously from streams of experience in longitudinal, lifelong settings.

REFERENCES

- [1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight Experience Replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [2] Lars Lien Ankile, Anthony Simeonov, Idan Shenfeld, Marcel Torne Villasevil, and Pulkit Agrawal. From imitation to refinement – residual RL for precise visual assembly. In *CoRL Workshop on Learning Robot Fine and Dexterous Manipulation: Perception and Control*, 2024.
- [3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.
- [4] Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774, August 2024. ISSN 1476-4687. doi: 10.1038/s41586-024-07711-7.
- [5] Perry Dong, Suvir Mirchandani, Dorsa Sadigh, and Chelsea Finn. What matters for batch online reinforcement learning in robotics? *arXiv*, 2025.
- [6] Mohamed Elsayed, Gautham Vasan, and A. Rupam Mahmood. Streaming deep reinforcement learning finally works. *arXiv preprint arXiv:2410.14606*, 2024.
- [7] Tyler L Hayes, Giri P Krishnan, Maxim Bazhenov, Hava T Siegelmann, Terrence J Sejnowski, and Christopher Kanan. Replay in deep learning: Current approaches and missing biological elements. *Neural Comput.*, 33(11):2908–2950, October 2021.
- [8] Physical Intelligence. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025.
- [9] Khurram Javed and Richard S. Sutton. The big world hypothesis and its ramifications for artificial intelligence. In *Finding the Frame: An RLC Workshop for Examining Conceptual Frameworks*, 2024.
- [10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR (Poster)*, 2015.
- [11] Timothée Lesort, Vincenzo Lomonaco, Andrei Stoian, Davide Maltoni, David Filliat, and Natalia Díaz-Rodríguez. Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, 58:52–68, 2020. ISSN 1566-2535. doi: <https://doi.org/10.1016/j.inffus.2019.12.004>.
- [12] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020.
- [13] Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 23190–23211. PMLR, 23–29 Jul 2023.
- [14] Shangke Lyu, Xin Lang, Han Zhao, Hongyin Zhang, Pengxiang Ding, and Donglin Wang. RL2ac: Reinforcement learning-based rapid online adaptive control for legged robot robust locomotion. In *Robotics: Science and Systems*, 2024.
- [15] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumar, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015. ISSN 1476-4687. doi: 10.1038/nature14236.
- [16] Aneesh Muppidi, Zhiyu Zhang, and Heng Yang. Fast TRAC: A parameter-free optimizer for lifelong reinforcement learning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [17] Evgenii Nikishin, Junhyuk Oh, Georg Ostrovski, Clare Lyle, Razvan Pascanu, Will Dabney, and Andre Barreto. Deep Reinforcement Learning with Plasticity Injection. *Advances in Neural Information Processing Systems*, 36:37142–37159, December 2023.
- [18] Octo Model Team. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- [19] Xiatao Sun, Shuo Yang, Yinxing Chen, Francis Fan, Yiyan Liang, and Daniel Rakita. Dynamic rank adjustment in diffusion policies for efficient and flexible training. 06 2025. doi: 10.15607/RSS.2025.XXI.159.
- [20] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018. ISBN 0262039249.
- [21] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *Robotics: Science and Systems*, 2025.
- [22] Gautham Vasan, Mohamed Elsayed, Alireza Azimi, Jiamin He, Fahim Shariar, Colin Bellinger, Martha White, and A. Rupam Mahmood. Deep Policy Gradient Methods Without Batch Updates, Target Networks, or Replay Buffers. In *Advances in Neural Information Processing Systems*, volume 37, pages 845–891, 2024.